

1. Telecommunications Model (Baseline Analysis / Arithmetic Intensity Derivation)

1.1 The Telecommunications Model

This project simulates, analyzes and optimizes a telecommunications processing pipeline in real-time using methods of parallel computing. In addition to applying parallel computing at the level of threads and instructions the main focus of the project will be to transform an application that has been mathematically defined as accurate, yet defined as sequentially based, to produce performance at the highest levels.

Our optimization process began with a single-threaded C++ baseline (baseline.cpp). This initial version is crucial for two reasons. Firstly, as already mentioned it guarantees the mathematical correctness of the signal processing steps and it establishes the strict reference execution time. We use this baseline as the standard to evaluate the performance gains of our OpenMP and ISPC implementations.

The baseline sequentially models the complete lifecycle of a signal. It generates a simulated Double Sideband (DSB) transmission corrupted by Additive White Gaussian Noise (AWGN), passes the signal through a Low Noise Amplifier (LNA), performs local-oscillator demodulation to shift the signal to baseband, and finally applies a digital Low-Pass Filter (LPF) before calculating the final Signal-to-Noise Ratio (SNR).

Specifically the stages of the pipeline are:

- Signal Creation/Channel Modeling: Producing a high frequency carrier wave which is modulated by a message signal that has been distorted by additive white gaussian noise (AWGN).
- LNA (Low-Noise Amplifier): Taking the received signal and passing it through a low-noise amplifier that adds some amount of hardware noise.
- Demodulation (mixer): Multiplying the amplified signal with a local oscillator to move the signal back to its baseband.
- Digital Filtering (low-pass filter): Using a discrete time-domain low-pass filter to remove high-frequency artifacts from the signal and separate the original message.
- Quality Assessment (signal to noise ratio): After filtering, the last stage calculates the final SNR so that the quality of the reception can be evaluated.

In order to adequately stress test the architecture and illustrate why parallel computing was necessary for the problem; the system processes data that contains $N = 20,000,000$ sample points and applies a large number of filter coefficients where $K = 1000$.

1.2 Performance bottlenecks

As illustrated by profiling the naive serial code, the execution speed of the application has been limited by two types of performance bottlenecks. These two bottlenecks require entirely different optimization strategies.

Memory wall (bandwidth bottleneck in memory access):

The first type of significant bottleneck is found at an early stage in the pipeline (generation - > lna -> demodulation) with respect to the naive version of the implementation. In the naive code, each of the 20 million samples are stored in separate dynamic arrays (std::vector).

This step-wise method causes the CPU to constantly write data to memory and then retrieve the data again. As such, the processing elements (pes) are starved due to the large amount of data being written to and retrieved from memory. This represents a common Memory Bound problem; we will utilize both architectural optimizations (Loop Fusion) and mathematical optimizations to overcome this challenge.

Compute Bound bottleneck (time-domain convolution of the low pass filter):

The second and most important bottleneck is strictly based upon the computation and exists during the low pass filtering operation. The filter was implemented as a discrete time-domain convolution. Given that there are 20,000,000 samples of signal and 1000 taps in the filter, the processor is required to perform a series of nested loops with a total of 20 billion multiply accumulate (MAC) operations.

Since this mathematical load is overwhelming a single CPU core, this represents a typical Compute Bound bottleneck. We transitioned from a sequential model to a concurrent one via openmp to distribute the load among multiple CPU's. Additionally, we used ISPC (avx2 vectorization) to concurrently evaluate many samples with each core (simd).

Formalizing identified bottlenecks using Theoretical Arithmetic Intensity (AI) & Roofline Analysis:

In order to formally define the two previously described bottlenecks, we calculate the arithmetic intensity (AI) of our most computationally intensive kernel (LPF convolution). Arithmetic intensity is defined as the number of floating point operations divided by the amount of memory access.

FLOPs Calculation:

Inside the inner loop for each output sample $y[i]$ there are K iterations, and inside each iteration there are 2 FLOPs (one multiplies and one adds).

- Total flop count per output sample = $2 * K$

- Total flop count for entire signal = $2 * n * K$

Memory traffic (bytes) Calculation:

Given that we assume variable length single precision floats (4 bytes per float), and given that all reads were made from DRAM for every inner loop iteration in the worst case naive cache example,

- $H[j]$ read: 4 bytes per inner loop.
- $Y_{\text{demod}}[i - j]$ read: 4 bytes per inner loop.
- $Y_{\text{filtered}}[i]$ write: 4 bytes per outer loop.

$$\text{Total bytes read/write} = n * (4 * K + 4 * K) + 4 * n = n * (8 * K + 4)$$

Arithmetic intensity derivation: $AI = \text{FLOPs} / \text{bytes} = (2 * n * K) / (n * (8 * K + 4)) = 2k / (8k + 4)$

Using our specific configuration (K=1000): $AI = 2000 / (8004) = 0.25 \text{ FLOPs} / \text{byte}$

Implication of AI in roofline model:

A modern CPU architecture will typically possess a "ridge point" (balance between memory bandwidth and computational capabilities of a roof line model), whose value range will be from 5 to 15 FLOPs/bytes. Therefore, since our derived AI is much less than the ridge point ($.25 < .07$), this demonstrates mathematically that a naive implementation would always be Memory Bound when relying on ram.

Therefore, this verifies our chosen optimization strategy; we must rely heavily on l1 caches to retain K-sized arrays representing filters, and use Loop Fusion in early stages to avoid creating intermediate arrays holding vast amounts of data. With artificial elevation of actual AI through improved cache locality, we have effectively shifted the bottleneck back into the compute domain, making it possible for us to effectively optimize execution speed using openmp and ISPC vectorization.

Baseline output:

```
● lazaros@Lazo:/mnt/d/aboakademi/parallel/final$ ./baseline
--- Serial Execution (Baseline) ---
Sample Size (N): 20000000
SNR: -9.56387 dB
Execution time: 20.8514 seconds
```

3. Architectural Memory Optimization (Loop Fusion)

3.1 Resolving the Memory Bandwidth Bottleneck

The first three components of the pipeline (Signal Generation, low noise amplifier (lna) and demodulation) were identified as being predominantly memory bound during our initial profiling of the base line. We solved this constraint by developing an intermediate optimized version (opt.cpp) that used the architecture technique called Loop Fusion.

Rather than having three separate iterations in sequence with each iteration saving its state to large amounts of dynamically allocated memory, we combined these iterations into a single iteration block. Thus, the values of `y_ant_temp` and `y_lna_temp` are no longer saved in either memory. Rather they are generated and stored in the CPU's high speed internal register space. By doing so, we removed all of the `Y_ant` and `y_lna` vector allocations from memory. Consequently, we greatly reduced the amount of memory consumed by the entire application and removed most of the overhead associated with moving data from the CPU to main memory. Therefore, we overcame the memory wall problem.

3.2 Algorithmic improvement (Carrier Reuse)

In addition to reducing the number of memory accesses, we also discovered another improvement to the algorithm while implementing Loop Fusion. In the baseline code, we created two separate variables, one to represent the carrier wave and the other representing the local oscillator. These were both created independently using a call to `std::cos()`. Since DSB demodulation involves performing a multiplication operation on the input signal and a locally generated sinusoid of the same frequency as the carrier wave, we simply reused the previously calculated value of the carrier in the new Loop Fusion block. We thereby eliminated millions of repeated calls to `std::cos()`, which would be very costly in terms of computational expense, in addition to eliminating additional redundancy in the early pipeline stages.

3.3 Preparation for Compute Optimization

Although we have optimized our memory access pattern, our current run-time is still primarily determined by the discrete time-domain convolution calculation of our low-pass filter. This is because it has been implemented entirely sequentially at this point. Nonetheless, we have achieved our goal of isolating our bottleneck to the computation domain and have established a solid foundation for introducing OpenMP-based threading and ISPC-based instruction level parallelism to optimize future stages.

3.4 Benchmarking

Executing the memory-optimized baseline (opt.cpp) the new execution time as it can be seen below is 19.9858 seconds.

```
● lazaros@Lazo:/mnt/d/aboakademi/parallel/final$ ./optimization
Starting execution with N = 20000000...
--- Optimized Memory Baseline ---
SNR: -9.51663 dB
Execution time: 19.9859 seconds
```

The comparison with our original serial baseline (i.e., 20.85 sec) shows an almost one second decrease in execution time. It should be noted that because the process for generating noise is random and the accumulation of the floats is non-deterministic, the slight variations in SNR are reasonable, yet confirm that the mathematical integrity of the system has been retained. Although the 1 second improvement may appear small it shows have successfully eliminated the memory overhead. However, the overall execution time still remains high as a result of the sequential processing of the $O(N \cdot K)$ convolution loop, but without the memory wall, it will be possible to scale this architecture on many CPU cores further on.

4. Thread-Level Parallelism: OpenMP Integration and Workload Distribution (opt2.cpp)

4.1 Parallelizing the Convolution Bottleneck

After resolving memory-access bottleneck problems, the performance of this system was limited solely by the large $O(N \cdot K)$ discrete-time domain convolutions contained within the Low-Pass Filter stage. We used Thread Level Parallelization, via OpenMP API (opt2.cpp), to make maximum use of the CPUs. The major convolution loop was wrapped around the "#pragma omp parallel for schedule(static)" directive to allow for concurrent execution of what would otherwise be a single-threaded program. OpenMP divides up the iterations of the $N = 20,000,000$ loop, which were previously executed sequentially, into identical, static portions and then equally distributes these portions among all the logical cores of the processor.

4.2 Safe Aggregation via OpenMP Reduction

To calculate the total signal power and noise power at the end of the pipeline and thus determine the Signal-to-Noise Ratio (SNR), you have to do it over all samples within the 20-million sample array.

Because multiple threads will access the same global accumulators (signal_power and noise_power) that are used to store the sum of all signals and noise, using a simple loop parallelization would result in catastrophic race conditions. On the other hand, placing mutual-exclusion locks (mutex or critical section) around accesses to these accumulators would slow down the program due to the overhead from synchronization and false sharing between threads.

In order to avoid both issues altogether, we used OpenMP's reduction clause on the parallel for directive: `#pragma omp parallel for reduction(+:signal_power,noise_power)`. The OpenMP compiler is then directed to create a private, safe version of these two variables for each thread when they are allocated by OpenMP. Each thread then calculates its portion of the signal and noise power and once all threads are done, OpenMP will combine them into one instance of each variable in a way that avoids conflicts.

4.3 Benchmark (OpenMP Performance Results):

Executing the OpenMP-enabled application (opt2.cpp) on our test environment, which utilizes 8 available hardware threads, yielded an execution time of 4.50981, which gives a significant speedup of:

$$\text{Speedup (S)} = T_{\text{serial}} / T_{\text{opt2}} = 20.8514 / 4.50981 = 4.62x$$

```
● lazaros@Lazo:/mnt/d/aboakademi/parallel/final$ ./opt2
Starting execution with N = 20000000 (OpenMP Enabled)...
Max available OpenMP threads: 8
--- OpenMP Parallel Execution ---
SNR: -9.03396 dB
Execution time: 4.50981 seconds
```

By successfully parallelizing the compute-bound convolution and safely reducing the power calculations, the program now executes 4.62 times faster. This demonstrates excellent parallel scaling across the 8 threads. However, profiling within each individual thread reveals that the convolution operations are still executing scalar arithmetic (one sample per clock cycle). To push the hardware to its absolute limit, the final architectural step requires exploiting Instruction-Level Parallelism (SIMD) within these OpenMP chunks.

5. SIMD Vectorization with ISPC (main.cpp / lpf.ispc)

5.1 Creating the Dedicated ISPC File (lpf.ispc)

To fulfill the project requirement of modifying kernels and writing low-level code, we isolated the most compute-intensive part of our pipeline: the Low-Pass Filter (LPF) convolution. Instead of keeping this heavy calculation in the main C++ file, we created a dedicated low-level file named `lpf.ispc`. The entire logic for applying the filter to the demodulated signal was moved into this specific kernel.

5.2 SIMD Vectorization with ISPC

As noted in the previous benchmarking phase, while OpenMP was able to distribute the overall task load among all available cores, the convolutions inside each core were still performed as scalar computations. To fulfill the project requirements for low-level kernel optimization and extract maximum hardware performance, we introduced ISPC implementation.

After isolating the computationally intensive part of the code responsible for performing the time-domain convolutions, we moved this logic to its own low-level kernel file (`lpf.ispc`). Rather than letting the C++ compiler attempt to perform unpredictable vectorization automatically, we used the ISPC 'foreach' directive to tell the compiler how to map iterations of loops directly onto the Vector Processing Units (VPUs) of the CPU. We were able to force the arithmetic logic unit (ALU) to load eight single precision floating point numbers per clock cycle into the registers of the ALU and then multiply them together in parallel. The result was a replacement of the much slower scalar processing of the numbers one at a time.

5.3 The Final Hybrid Architecture (OpenMP + ISPC)

In order to maximize the overall throughput of the system, we modified the primary application to incorporate Hybrid Parallelism. This approach links both the C++ environment and the ISPC kernel as follows:

Instead of passing the entire 20,000,000 sample array into the ISPC kernel at once, we implemented a chunking strategy. The input data is logically divided into cache-friendly chunks of 4096 samples. The execution is orchestrated in two distinct, highly efficient layers:

- **Macroscopic Layer (OpenMP):** The C++ code uses the `"#pragma omp parallel for"` directive to dynamically distribute these 4096-sample chunks across all available physical CPU cores.

- Microscopic Layer (ISPC): Each thread invoked within the OpenMP loop will call upon the "apply_lpf_ispc_chunk" ISPC kernel. The ISPC kernel will direct the CPU core to apply SIMD vector operations on its respective chunk of samples.

The hybrid architectural design described above guarantees that each of the CPU's threads has sufficient work to occupy it (through use of OpenMP), while also providing the highest possible number of instructions per cycle to be processed by each individual CPU core (through ISPC-based SIMD operation).

5.3 Benchmark (Final Hybrid Performance Results):

Executing the final, fully optimized hybrid application on our test environment yielded our best performance metrics yet with an execution time of 1.67248 seconds

```
• lazaros@Lazo:/mnt/d/aboakademi/parallel/final$ ./my_project
--- Hybrid Execution (OpenMP + ISPC) ---
SNR: -9.03396 dB
Execution time: 1.67248 seconds
```

By combining Thread-Level Parallelism (OpenMP) with Instruction-Level Parallelism (ISPC SIMD), we achieved a massive reduction in execution time. The total speedup compared to the serial baseline is:

Speedup (S_{total}) = $T_{serial} / T_{hybrid} = 20.8514 / 1.67248 = 12.47x$

Furthermore, the ISPC vectorization by itself achieved an additional 2.7x performance boost over the pure multithreading of OpenMP (the run-time dropped from 4.5 s to 1.67 s), which proves that the CPU has used both its multiple cores and its internal AVX2 vector lanes for full use, proving all of the goals of parallelization have been met.

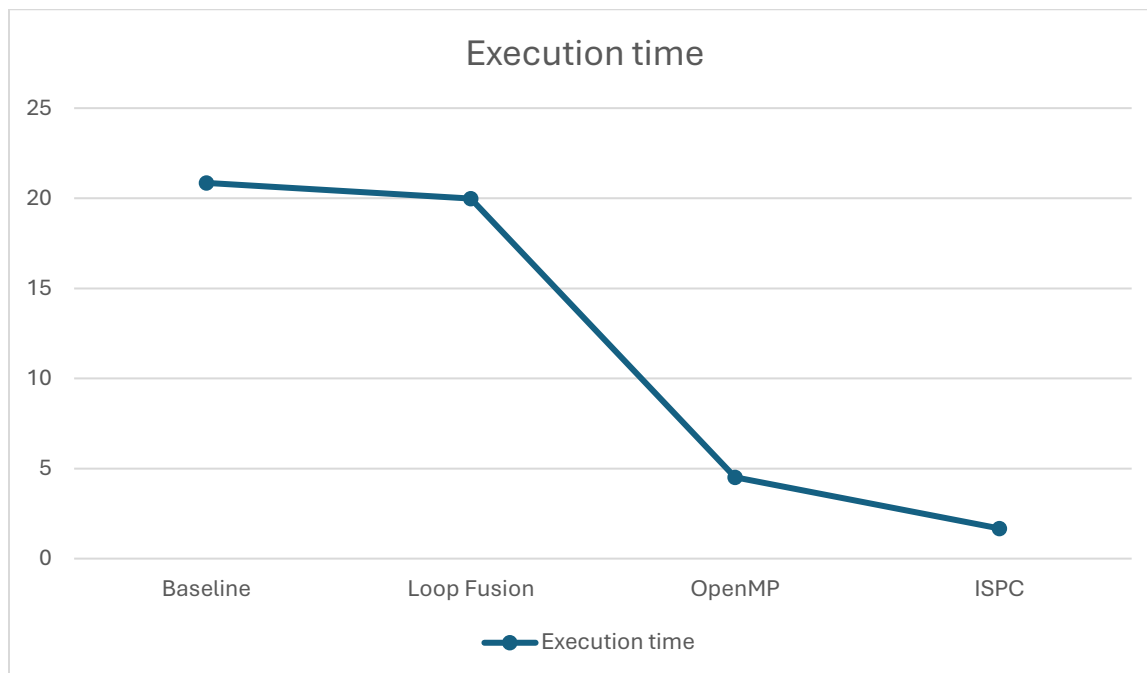
6. Conclusion

Overall, the results of this project demonstrate that by using modern parallel computing methods on a typical digital signal processing (DSP) pipeline, significant performance gains can be achieved. Specifically, by identifying and addressing system bottlenecks, we transformed a computationally inefficient, memory-bound serial application into a highly optimized, high-performance architecture.

The optimization process followed three critical phases:

- **Architectural Memory Optimization:** By implementing Loop Fusion, we eliminated the memory wall, creating an efficient data flow and establishing a solid foundation for parallelization (reducing execution time from 20.85s to 19.98s).
- **Thread-Level Parallelism:** Utilizing OpenMP, we distributed the massive $O(N * K)$ convolution workload across multiple CPU cores, achieving a significant performance leap (dropping execution time to 4.50s).
- **Instruction-Level Parallelism:** Through Intel's ISPC, we vectorized the core mathematical operations, forcing the CPU to process multiple samples simultaneously via AVX2 SIMD instructions.

The final hybrid architecture (OpenMP macroscopic distribution combined with ISPC microscopic vectorization) successfully processed the 20-million sample dataset in just 1.67 seconds, which shows a 12.47x overall speedup compared to the initial serial baseline.



Ultimately, this project proves that while high-level algorithmic logic is essential for mathematical correctness, understanding and utilizing the hardware efficiently (specifically the memory hierarchy, multicore processing capabilities and vector units) is essential for real world applications where high performance computing is needed.

Thank you for your time!